

JAVA SDE-2 INTERVIEW PREPARATION SERIES

INTERVIEW ROUNDS - BOOK 01 OF 01 - STUDY STEP IR-01

Behavioral and Leadership Interviews

Story Banks, STAR Delivery, and the Signals That Decide the SDE-2 Level

BY

Vinay Reddy Kalluri

Prepare the round that most often decides whether a strong engineer is offered SDE-2 or SDE-1: build a story bank that covers every signal twice, deliver answers where your decisions are unmistakable, and map your stories onto any company's values.

SIGNALS | STORY BANK | STAR | CONFLICT | FAILURE | MENTORING | LEADERSHIP PRINCIPLES

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [Book 01 in the Interview Rounds segment](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This book covers the behavioral and hiring-manager rounds: what they score at SDE-2, how to build and audit a story bank, how to structure and time answers, the signal-specific stories interviewers ask for, and how to adapt one bank to company-specific values such as Amazon's Leadership Principles.

Readiness map

Decision	Evidence
START HERE	A question begins with 'tell me about a time'; The round is labelled behavioral, values, leadership, or hiring manager; The company publishes values or leadership principles; You passed the technical rounds but were offered a lower level before.
PREPARE FIRST	At least two years of professional engineering experience to draw stories from; No technical prerequisites; the examples are drawn from backend engineering; Access to your own design documents, post-mortems, and reviews helps.
MOVE ON WHEN	Name the signals an SDE-2 behavioral round scores and what separates SDE-2 from SDE-1 evidence; Build a story bank that covers every signal at least twice, and plan a loop without forced reuse; Deliver STAR answers with at least 55% on your actions, in one and a half to three minutes; Tell conflict, failure, and mentoring stories that survive five levels of probing; Map one story bank onto a company's values, including all sixteen Amazon Leadership Principles.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Interview Rounds Segment Position

START OF SEGMENT	YOU ARE HERE IR 01 - Behavioral and Leadership Interviews	SEGMENT COMPLETE
------------------	---	------------------

A note from Vinay

You already have the experience this round is looking for; what most engineers lack is the preparation to show it. Write your stories down, check that every signal has two of them, and practise saying what you decided and what you rejected. Tell your own stories, including the failures - they are the ones that survive the follow-up questions.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - What the Behavioral Round Scores	4
Chapter 2 - Building a Story Bank	9
Chapter 3 - Structuring and Delivering an Answer	14
Chapter 4 - Ownership, Ambiguity, and Delivery	19
Chapter 5 - Conflict, Disagreement, and Influence	23
Chapter 6 - Failure, Feedback, and Growth	27
Chapter 7 - Mentoring and Leadership Without Authority	32
Chapter 8 - Company Formats and Leadership Principles	36
Chapter 9 - The Hiring-Manager Round and Closing Well	41
Chapter 10 - Behavioral Interview Drills	46
Chapter 11 - Behavioral Interview Drills: Worked Solutions	48
Chapter 12 - Executable Story Bank Model	52
Part II - Practice and Handoff	63

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - What the Behavioral Round Scores

Learning objectives

By the end of this chapter, you should be able to:

- explain why behavioral interviews ask about the past instead of hypotheticals;
- name the signals an SDE-2 behavioral round is scored on, and how they differ from SDE-1;
- recognise the answers that get a strong engineer down-levelled;
- describe how interviewers probe beneath a rehearsed story; and
- decide how much preparation time the round deserves relative to coding and design.

WHY IT WORKS | Why this matters at SDE-2

Engineers routinely under-prepare this round because it feels like conversation. It is not. At most large companies the behavioral round, or the behavioral half of a hiring-manager round, is scored against written criteria with the same weight as a coding round, and it is the round most often cited when a candidate who passed every technical round is offered SDE-1 instead of SDE-2. The reason is simple: coding and design rounds show what you can do; the behavioral round is the main evidence of *the scope at which you already operate*, and scope is what separates levels.

WHY IT WORKS | First-principles model

The premise of behavioral interviewing is that **past behaviour in a specific situation is the best available predictor of future behaviour in a similar one**. Hypothetical questions ("what would you do if...") invite the answer the candidate knows is correct. Questions about a real past event ("tell me about a time when...") invite evidence, and evidence can be probed.

So the round is an evidence-gathering exercise. The interviewer's job is to collect specific, first-person examples of the signals on their rubric, at a level of detail that could not easily be invented. Your job is to supply that evidence efficiently: the right story for each signal, told so that your own decisions are unmistakable.

Specification boundary: every company writes its own rubric and few publish it in full. Amazon publishes its Leadership Principles; most others describe their values more loosely. The signals in this book are the common core that appears, under different names, in almost every SDE-2 rubric. Always read the preparation material your recruiter sends; where it names specific values, map your stories onto those names.

Core terminology

- **Signal:** a behaviour the rubric scores, such as ownership or handling conflict.
- **Level calibration:** judging whether evidence matches the expected scope for the level.
- **Scope:** the size of the problem you owned - a task, a feature, a system, a team's direction.
- **Probe (drill-down):** a follow-up question that tests whether a story's details hold up.
- **Down-level:** an offer at a lower level than the one interviewed for.
- **Bar raiser:** at Amazon, an interviewer from outside the hiring team whose role is to hold the hiring standard.

Detailed mechanics

The signals

Twelve signals cover nearly every SDE-2 behavioral question. The companion model uses exactly these:

Signal	What the interviewer wants to see
Ownership	You treated an outcome as yours beyond your assigned task
Ambiguity	You made progress when the problem or goal was unclear
Technical depth	Your judgement rested on understanding, not guesswork
Delivery	You shipped against real constraints
Conflict	You disagreed productively and the relationship survived
Influence	You changed a decision you did not control
Failure	You made a real mistake, owned it, and changed
Feedback	You gave or received hard feedback and acted on it
Mentoring	You made someone else more effective
Customer focus	Your decisions traced back to a user or customer need
Prioritization	You chose what not to do, and said so
Incident	You led or significantly contributed under production pressure

What changes at SDE-2

The same question is scored differently at different levels. For "tell me about a time you owned something end to end":

- **SDE-1 evidence:** owned a well-defined feature; asked good questions; delivered it well.
- **SDE-2 evidence:** owned an ambiguous problem across more than one component or team; defined the approach; managed risk; the outcome changed something measurable for others.

The pattern generalises. At SDE-2 the interviewer looks for problems you *framed* rather than were handed, decisions whose trade-offs you can articulate, and influence that reached beyond your own code - into another team's design, a process, or a teammate's growth.

Why strong engineers get down-levelled

Four patterns account for most down-levels in this round:

- 1 **Scope too small.** Every story is a task someone else defined. Even excellent execution reads as SDE-1.
- 2 **"We" throughout.** The interviewer cannot tell what *you* did, so they cannot credit it.
- 3 **No trade-offs.** The story has one path and it worked. Without rejected alternatives there is no evidence of judgement.
- 4 **Unreal failure.** "My weakness is that I care too much" or a failure that was secretly someone else's. Interviewers read this as low self-awareness.

Each of these is fixable in preparation, and later chapters address them directly.

How probing works

A good interviewer spends a minute or two on your story and the rest of the time probing it:

- "What exactly did *you* do, as opposed to the team?"
- "What else did you consider? Why not that?"
- "How did you know it worked? What was the number?"
- "What would you do differently?"
- "What did the other person say when you pushed back?"

Probes are not a sign that the answer was weak; they are how the interviewer turns a story into scored evidence. A story you really lived survives five levels of "why"; a story assembled from someone else's project collapses at the second. This is the strongest argument for telling your own stories, even when they feel smaller than an invented one.

How much to prepare

Behavioral preparation has a very high return per hour. A bank of eight to ten well-chosen stories, each rehearsed aloud a few times and checked against the probe questions, covers most behavioral loops. Plan roughly one hour per story for the first draft and rehearsal, plus mock rounds - about the time you would spend on one DSA topic, for a round that often carries the weight of a full coding round.

WATCH OUT | Failure modes and common mistakes

- **Treating it as small talk.** It is scored against criteria with real weight.
- **Answering hypothetically.** "I would..." when asked "tell me about a time" gives no evidence.
- **Only SDE-1-scope stories.** Well-told tasks still read as SDE-1.
- **Telling someone else's story.** It fails at the second probe.
- **Refusing to name a failure.** Reads as low self-awareness, the opposite of the intent.

TRY IT | Interview questions and model answers

These are the meta-questions candidates ask about the round, answered as a hiring panel would.

Q: Why not just ask what I would do in a situation? A: Because everyone knows the right answer to a hypothetical. A real past event can be probed for specifics - what you did, what you rejected, what happened - and those specifics are the evidence the rubric scores.

Q: What makes a behavioral answer SDE-2 rather than SDE-1? A: Scope and judgement. The problem was ambiguous or crossed boundaries, you framed the approach, you can name the alternatives you rejected and why, and the result mattered to people beyond you.

Q: Is it bad if the interviewer keeps interrupting with questions? A: Usually the opposite. Probing is how interviewers collect evidence; a real story with real detail gives them plenty to score.

TRY IT | Exercises

- 1 List the three largest things you have owned in the last two years. For each, write one sentence stating the scope - task, feature, system, or cross-team.
- 2 For each of the twelve signals, write the name of one story that could evidence it. Mark the signals where you have nothing.
- 3 Take one story and answer the five probe questions in writing. Which one was hardest?

RECAP | Chapter summary

Behavioral rounds collect probe-able evidence of past behaviour against a rubric of about a dozen signals. At SDE-2 the same signals are held to a larger scope: ambiguous problems you framed, trade-offs you can defend, and influence beyond your own work. Most down-levels come from small scope, "we" answers, missing trade-offs, and unreal failures - all fixable in preparation.

TRY IT | Revision checklist

- ☐ I can name the twelve signals and what each one needs to show.
- ☐ I can explain the SDE-1 versus SDE-2 difference for an ownership story.
- ☐ I can list the four down-levelling patterns.
- ☐ I have answered the five probe questions for at least one story.

SDE-2 CORE CHAPTER

Chapter 2 - Building a Story Bank

Learning objectives

By the end of this chapter, you should be able to:

- mine your own history for candidate stories systematically instead of from memory on the day;
- choose eight to ten stories that together cover every signal at least twice;
- record each story in a compact, reusable format;
- plan which story answers which question in a loop without over-using one; and
- find and fill the gaps before an interviewer does.

WHY IT WORKS | Why this matters at SDE-2

Candidates who prepare behavioral rounds by rehearsing answers to specific questions run out of material: there are hundreds of phrasings and only a few real experiences. Candidates who prepare a *story bank* - a small set of rich stories tagged with the signals each can evidence - can answer almost any phrasing by choosing a story and emphasising the right part. The bank also exposes gaps in advance, which is the only time they can be fixed.

WHY IT WORKS | First-principles model

A story bank is a **many-to-many mapping from stories to signals**. One rich story evidences several signals; each signal should be covered by at least two stories, because a loop may ask about the same signal twice and one story cannot be used for every round.

Treat the bank as data. The companion model does exactly that: it tags stories with signals, prints the coverage, and flags any signal with fewer than two stories.

Core terminology

- **Story bank:** the set of prepared stories, each tagged with signals.
- **Coverage:** the number of stories that evidence a given signal.
- **Anchor story:** a large, rich story that covers four or more signals.
- **Loop plan:** the assignment of stories to the questions you expect across a day of interviews.
- **Gap:** a signal with no story, or with only one.

Detailed mechanics

Mining for stories

Do not start from the questions. Start from your history, because that is where the evidence is:

- 1 **Walk your resume backwards**, project by project, for the last three to five years.
- 2 **For each project, list the moments**: a decision you made, a disagreement, something that broke, something you changed your mind about, someone you helped.
- 3 **Check your written trail**: design documents, incident post-mortems, performance reviews, and long pull-request threads. They contain specifics - numbers, dates, names of alternatives - that memory has blurred.
- 4 **Keep everything at first**, then choose.

A useful target is twenty candidate moments, from which you will choose eight to ten stories.

Choosing the bank

Prefer stories that are:

- **Recent** - within about three years, so that the scope reflects your current level.
- **Yours** - you made the key decisions, even if a team delivered the result.
- **Measurable** - there is a number: latency, incidents, time saved, revenue, adoption.
- **Rich** - an anchor story covers several signals and survives deep probing.
- **Varied** - different projects, teams, and kinds of problem.

The sample bank and its gaps

The companion's sample bank is a composite for a backend engineer with about five years of experience: eight stories, including a database migration (S1), leading an outage response (S2), a design disagreement (S3), a self-inflicted bug (S4), a cross-team library adoption (S5), mentoring through on-call (S6), a scope cut for a regulatory deadline (S7), and hard design-review feedback (S8). Its coverage:

TEXT

OWNERSHIP	3 stories	[S1, S2, S4]
AMBIGUITY	2 stories	[S1, S5]
TECHNICAL_DEPTH	3 stories	[S1, S2, S3]
DELIVERY	2 stories	[S1, S7]
CONFLICT	1 story	[S3]
INFLUENCE	2 stories	[S3, S5]
FAILURE	2 stories	[S4, S8]
FEEDBACK	2 stories	[S6, S8]
MENTORING	1 story	[S6]
CUSTOMER_FOCUS	1 story	[S7]
PRIORITIZATION	1 story	[S7]
INCIDENT	2 stories	[S2, S4]

signals with fewer than two stories: [CONFLICT, MENTORING, CUSTOMER_FOCUS, PRIORITIZATION]

Every signal is covered, which looks fine at a glance. The model says otherwise: four signals rest on a single story. When I first wrote the expected gaps for this chapter, I missed conflict - the bank has an obvious disagreement story, so it felt covered. But conflict is one of the most frequently asked signals, and with only one story, a second conflict question in the same loop has no fresh answer. The fix is to add a story, not to stretch S3 twice.

The story card

Record each story on one card, in notes rather than prose:

TEXT	
ID / title:	S1 - Migrated order service off a shared database
When / scope:	2024, 2 services, 2 teams, 1 quarter
Situation:	shared DB; schema changes needed both teams; 2 checkout incidents / quarter
My decisions:	mapped 14 cross-boundary queries; replaced 3 joins with an API; 4-step reversible migration; stopped the backfill on a 2% mismatch
Alternatives:	shared read replica (rejected: coupling stays); big-bang cutover (rejected: risk)
Conflict:	checkout lead worried about latency -> prototype -> +4 ms p99, agreed
Result:	2 weeks early, no downtime, 0 DB incidents in 2 quarters
Learning:	start data comparison earlier (found bug late, cost a week)
Signals:	ownership, ambiguity, technical depth, delivery (+ influence via the prototype)

A card is for recall under pressure. The numbers and the rejected alternatives are the parts people forget and interviewers probe.

Planning a loop

A typical SDE-2 loop asks five to eight behavioral questions across several interviewers, who often compare notes afterwards. Telling the same story to every interviewer wastes the loop's evidence and looks thin. Plan it:

TEXT	
Q1 OWNERSHIP	-> S2
Q2 CONFLICT	-> S3
Q3 FAILURE	-> S8
Q4 INCIDENT	-> S4
Q5 INFLUENCE	-> S5
Q6 OWNERSHIP	-> S1

The companion's planner assigns each question the least-used story that covers it, with a limit of one use per story. This six-question loop is covered with no reuse. A different loop is not:

TEXT
mentoring, feedback, mentoring again -> [S6, S8, GAP]

A second mentoring question finds no unused story - the same gap the coverage table predicted. In practice, you may reuse a rich story for a different signal with a different emphasis, but you should choose to, not be forced to.

Filling gaps honestly

A gap does not mean you lack the experience. It usually means you have not noticed which of your experiences counts. For mentoring: onboarding a new hire, reviewing an intern's design, pairing through someone's first on-call. For customer focus: a time you pushed back on a requirement because of what users actually did. If there is genuinely nothing, say so in the round and describe what you are doing about it. Do not invent a story; it will not survive probing.

WATCH OUT | Failure modes and common mistakes

- **Preparing answers to questions instead of stories.** You will face phrasings you did not prepare.
- **Too few stories.** Five stories cannot cover a six-question loop without repetition.
- **One story per signal.** Looks covered; fails on the second question.
- **Stories without numbers.** The first probe asks "how did you know it worked?"
- **Old stories.** A strong story from six years ago shows the scope you had six years ago.

TRY IT | Interview questions and model answers

Q: Tell me about a time you had to work with ambiguous requirements. A strong answer comes from a card - S1 above: the goal was "stop the incidents", not a specification; the candidate framed it as a data-ownership problem, mapped the queries, and chose an approach with named alternatives.

Q: Can you give me another example? The test of a bank. Have a second story for every signal: here, S5, getting three teams to adopt a shared library when nobody had agreed what the problem was.

TRY IT | Exercises

- 1 Mine your last three years and list twenty candidate moments.
- 2 Choose eight to ten stories and fill in a card for each.
- 3 Tag each card with signals and build the coverage table. List every signal with fewer than two stories, and find a second story for each.
- 4 Plan a six-question loop that uses no story twice.

RECAP | Chapter summary

Prepare stories, not answers. Mine your history systematically, choose eight to ten recent, measurable stories that are genuinely yours, and record each on a card with its decisions, rejected alternatives, numbers, and learning. Treat the bank as data: every signal needs at least two stories, and a planned loop should not force reuse. The sample bank looked complete and still had four single-story signals, including conflict.

TRY IT | Revision checklist

- ☐ I have twenty candidate moments and eight to ten chosen stories.
- ☐ Every story has a card with numbers and rejected alternatives.
- ☐ Every signal has at least two stories.
- ☐ I have planned a loop with no forced reuse.

SDE-2 CORE CHAPTER

Chapter 3 - Structuring and Delivering an Answer

Learning objectives

By the end of this chapter, you should be able to:

- structure an answer with STAR and allocate time so that most of it is about your actions;
- keep an answer between about one and a half and three minutes;
- use "I" for your decisions and "we" for shared outcomes, and check the balance;
- make results concrete, and close with what you learned;
- handle probes, clarifying questions, and the moment you realise you picked the wrong story.

WHY IT WORKS | Why this matters at SDE-2

Two candidates with the same experience can score very differently on delivery alone. One spends ninety seconds on background and runs out of time before the decision that mattered. The other states the stakes in two sentences and spends a minute on what they decided and why. Delivery is not polish; it determines whether the evidence reaches the interviewer at all.

WHY IT WORKS | First-principles model

An answer has one job: **make your decisions and their consequences unmistakable, fast**. Everything else - context, team, outcome - exists to make those decisions understandable.

STAR is the standard shape:

- **Situation:** the context, in two or three sentences, including the stakes.
- **Task:** what you specifically owned.
- **Action:** what you did, decided, and rejected - the bulk of the answer.
- **Result:** what happened, measured, and what you learned.

Many interviewers also listen for a closing **Learning** (sometimes written STARL): what you would do differently, or what you now do by default.

Core terminology

- **STAR:** Situation, Task, Action, Result.
- **Setup share:** the fraction of the answer spent on situation and task.
- **Action share:** the fraction spent on your actions and decisions.
- **Probe:** an interviewer's follow-up question that tests a detail.
- **Headline:** a one-sentence summary said first, so the interviewer knows where the story is going.

Detailed mechanics

Allocating time

The companion measures a worked answer - the database migration from chapter 2 - section by section, at a speaking rate of 150 words per minute. That rate is an assumption: typical conversational speech is often quoted at around 130 to 160 words per minute, and nerves push it higher. Time yourself.

TEXT				
Situation	30 words	12.0%	12 s	
Task	22 words	8.8%	9 s	
Action	154 words	61.6%	62 s	
Result	44 words	17.6%	18 s	
total	250 words		100 s (1.7 min)	

The targets the model checks:

- **Action at least 55%** of the answer. The actions are the evidence.
- **Situation and task together at most 25%.** Context beyond what makes the decisions understandable is time taken from evidence.
- **One and a half to three minutes in total.** Shorter leaves no detail to score; longer loses the interviewer and leaves no time for probes, which is where much of the scoring happens.

The worked answer

Here is the answer the model measured, as it would be spoken. Notice what each part does.

Situation. Our checkout service and the order service shared one PostgreSQL database. Every schema change needed both teams, and a slow order query had caused two checkout incidents in one quarter.

Task. I owned the plan to give the order service its own database without downtime, and I had one quarter to do it.

Action. I started by mapping every query that crossed the boundary, and found fourteen, three of them joins that checkout depended on. I proposed replacing the joins with an order-summary API, and wrote a one-page design that listed the three options I had rejected and why. The checkout lead pushed back on latency, so I built a prototype and measured it: the API added four milliseconds at p99, which we agreed was acceptable. I then ran the migration in four steps, each reversible: dual writes behind a flag, a backfill with row-count and checksum comparison, reads switched one endpoint at a time, and finally removing the old tables. During the backfill I found that two percent of rows differed because of a timezone bug in an old job, so I stopped, fixed the job, and re-ran the comparison before continuing. I wrote a runbook for each step so that anyone on call could roll back.

Result. We finished two weeks early with no customer-facing downtime. Checkout had no database-related incidents in the next two quarters, and schema changes stopped needing cross-team review. I would now start the data comparison earlier, because finding the timezone bug late cost us a week.

What makes it strong: the stakes are in the first two sentences; the action section contains decisions (replace joins with an API, four reversible steps, stop the backfill), alternatives (three rejected options), a disagreement resolved with evidence (the prototype), and a number for each claim; the result is measured and followed by an honest learning.

"I" versus "we"

Interviewers score what *you* did. The companion counts first-person pronouns in the worked answer:

TEXT

whole answer: 11 singular, 4 plural
action section: 8 singular, 1 plural

The action section is overwhelmingly "I", because it describes the candidate's decisions. "We" appears where it is honest - the agreement with the checkout lead, and the team's shared result. That is the balance to aim for: "I" for what you decided and did, "we" for outcomes the team shares. A story told entirely in "we" cannot be credited to you; a story told entirely in "I" about a team effort sounds as though you are taking credit for others' work.

Start with a headline

Before the situation, say one sentence that tells the interviewer where you are going: "This is about moving a service off a shared database without downtime, where I had to convince another team." It lets the interviewer listen for the right things and redirect you immediately if the story does not fit the question.

Results with numbers - and without them

Prefer a number: latency, error rate, incidents, time, cost, adoption. If there is no number, use the next best evidence: "the other two teams adopted the library within a month", "the on-call runbook became the team template". If the result was bad, say so; a failed project with a clear learning can be a strong story.

Handling probes

- **Answer the question asked**, briefly, then stop. Probes are short questions wanting short answers.
- Say **"I don't remember the exact figure, but it was roughly..."** rather than inventing a precise number. Precision you cannot defend damages everything else you said.
- **Credit others precisely.** "The design was mine; the backfill tooling was built by a teammate" is a strong answer, not a weak one.

Picking the wrong story

If a minute in you realise the story does not fit the question, say so: "I realise this is more about delivery than conflict - would you like me to finish it, or switch to a better example?" Interviewers respect the self-awareness, and it saves the round.

WATCH OUT | Failure modes and common mistakes

- **Long setup.** Ninety seconds of context and thirty of action.
- **All "we".** Nothing can be credited to you.
- **No alternatives.** There is no evidence of judgement without the paths not taken.
- **No number in the result**, or an invented one.
- **Memorised word-for-word.** Sounds recited and breaks at the first probe. Rehearse the card, not a script.
- **Over five minutes.** The interviewer loses the thread and has no time to probe.

TRY IT | Interview questions and model answers

Q: Tell me about a project you're proud of. A: A headline, then a two-minute STAR answer like the one above: stakes in two sentences, your decisions and rejected alternatives in the middle, a measured result, and what you would do differently.

Q: What exactly was your part, versus the team's? A: Name the split precisely: "I wrote the design, built the prototype, and ran the migration steps; a teammate built the backfill tool, and the checkout team changed their callers to the new API."

Q: How do you know it worked? A: The number, and how it was measured: "no database-related checkout incidents in the two quarters after, from our incident tracker, against two in the quarter before."

TRY IT | Exercises

- 1 Write one story as a full STAR answer. Count the words in each section. Is action at least 55%?
- 2 Record yourself telling it. Time it, and count "I" and "we".
- 3 Rewrite the answer so that the stakes appear in the first two sentences.
- 4 Have someone ask you five probes and answer each in under twenty seconds.

RECAP | Chapter summary

Make your decisions unmistakable, fast. Open with a headline, spend no more than a quarter of the answer on setup and at least 55% on actions and decisions, and keep the whole answer to one and a half to three minutes. Use "I" for your decisions and "we" for shared outcomes, include the alternatives you rejected, measure the result, and close with a learning. Rehearse from the card, not a script, so that probes find real detail.

TRY IT | Revision checklist

- ☐ I can state the STAR time allocation and the total length target.
- ☐ My main stories each open with a headline.
- ☐ My action sections name rejected alternatives.
- ☐ My results have numbers, or honest substitutes.
- ☐ I know how to recover when I pick the wrong story.

SDE-2 CORE CHAPTER

Chapter 4 - Ownership, Ambiguity, and Delivery

Learning objectives

By the end of this chapter, you should be able to:

- tell an ownership story that shows scope beyond your assigned task;
- show how you made progress when the problem itself was unclear;
- describe a delivery under real constraints, including what you cut and why;
- present technical depth in a way a non-specialist interviewer can score; and
- recognise the weak versions of these stories and upgrade them.

WHY IT WORKS | Why this matters at SDE-2

Ownership, ambiguity, and delivery are asked in nearly every SDE-2 loop, often several times, and together they carry most of the "scope" evidence that decides the level. They are also the stories where engineers most often undersell themselves by describing the work instead of the judgement.

WHY IT WORKS | First-principles model

These three signals share one question: **did you treat an outcome as yours, and what did that make you do that nobody asked you to?**

- **Ownership** is visible in actions outside the assigned task: noticing a risk nobody owned, following a problem across a team boundary, fixing the process that caused the bug.
- **Ambiguity** is visible in how you created structure: turning "make checkout reliable" into a problem statement, a measurement, and a plan.
- **Delivery** is visible in trade-offs: what you cut, what you protected, and how you kept stakeholders informed.

Core terminology

- **Scope:** the size and boundary of what you owned.
- **Problem framing:** turning a vague goal into a specific, measurable problem.
- **Descoping:** deliberately removing work to protect a deadline or quality.
- **Stakeholder:** someone affected by, or with a say in, the outcome.
- **Leading indicator:** an early signal that a plan is on or off track.

Detailed mechanics

Ownership: beyond the ticket

Weak version: "I was assigned the migration and I completed it on time."

Strong version adds what ownership made you do that the ticket did not require:

- you noticed the rollback path was untested, and wrote and rehearsed runbooks;
- you found a data bug in someone else's job during the backfill, and fixed it rather than working around it;
- you measured the outcome after launch instead of closing the ticket.

The database migration from chapter 3 is an ownership story for exactly these reasons. The probe to prepare for: "What would have happened if you had not done that?"

Ambiguity: creating structure

Ambiguity questions ask for a time when the goal, the requirements, or the path was unclear. The evidence is the sequence you used to reduce it:

- 1 **Clarify the outcome.** Who is affected, and what would "done" look like? Write it down and get agreement.
- 2 **Measure the current state.** A number turns an opinion into a problem.
- 3 **Find the smallest step that reduces uncertainty:** a prototype, a spike, a data pull.
- 4 **Commit to a direction with an explicit checkpoint** for changing it.

In the sample bank, S5 - getting three teams to adopt a shared idempotency library - starts as "we keep having duplicate-payment bugs". The candidate counted the incidents across teams (the measurement), found that all of them came from hand-rolled retry logic (the framing), built a small library for one team first (the reducing step), and agreed a review point after one month (the checkpoint).

Delivery: trade-offs under constraint

Delivery stories are about the constraint - a deadline, a staffing gap, a dependency - and what you did because of it. The sample bank's S7 is a regulatory deadline that could not move. The evidence to include:

- **What you cut, and why that and not something else.** "We dropped the admin dashboard and kept the audit log, because the regulation required the log and support could query the data directly for a quarter."
- **What you refused to cut.** Usually tests, observability, or rollback.
- **How you communicated.** Who knew about the trade-off, and when - before the deadline, not after.
- **The result, including cost.** "We met the deadline; the dashboard shipped six weeks later."

A delivery story without a trade-off is just a schedule that worked.

Technical depth for a mixed panel

Behavioral interviewers are not always engineers in your area. Make depth scorable by stating the decision, the alternative, and the consequence in plain terms: "I replaced three cross-service joins with an API call. The alternative was a shared read replica, which would have kept the teams coupled. The API added four milliseconds, which I measured before we committed." An expert can probe further; a generalist can still score the judgement.

Upgrading weak stories

Weak signal	Upgrade question to ask yourself
"I completed the task"	What did I do that the task did not require?
"The requirements were unclear, so I asked"	What structure did I create after asking?
"We shipped on time"	What did we cut or protect to make that happen?
"It was technically complex"	What did I decide, instead of what, and how did I know?

WATCH OUT | Failure modes and common mistakes

- **Describing the work instead of the judgement.** The interviewer wants decisions.
- **Heroics.** Staying up all night is not ownership; preventing the next night is.
- **Ambiguity resolved by asking the manager.** Asking is fine; the evidence is what you did next.
- **Delivery with no trade-off.** Nothing to score.
- **Jargon a generalist cannot score.** Decision, alternative, consequence - in plain terms.

TRY IT | Interview questions and model answers

Q: Tell me about a time you took ownership of something outside your responsibilities. A: The duplicate-payment bugs (S5). They were spread across three teams and nobody owned the pattern. I counted eleven incidents in six months, traced every one to hand-rolled retries, built a small idempotency library with one team first, and after a month two more teams adopted it. Duplicate-payment incidents went to zero for the next two quarters. What I would do differently: involve the other teams' leads at the start instead of after the first adoption, which would have saved a month.

Q: Describe a time you had to make a decision without all the information. A: Choose a story where you committed with a checkpoint: what you knew, what you did not, why waiting was costlier than deciding, and what signal would have made you reverse it.

Q: Tell me about a time you had to deliver under a tight deadline. A: S7: the regulatory deadline, what was cut (the dashboard), what was protected (the audit log and its tests), who was told and when, and what it cost afterwards.

TRY IT | Exercises

- 1 Take your main ownership story and list three things you did that the task did not require. If you cannot, choose a different story.
- 2 Rewrite an ambiguity story as the four-step sequence above.
- 3 For a delivery story, write one sentence each for: what you cut, what you protected, who you told.

RECAP | Chapter summary

Ownership, ambiguity, and delivery carry most of the scope evidence at SDE-2. Show ownership through actions the task did not require, ambiguity through the structure you created, and delivery through the trade-offs you made and communicated. State technical depth as decision, alternative, and consequence so that any interviewer can score it.

TRY IT | Revision checklist

- ☐ My ownership story includes actions beyond the assigned task.
- ☐ My ambiguity story follows clarify, measure, reduce, commit.
- ☐ My delivery story names what was cut, what was protected, and who was told.
- ☐ I can state a technical decision so that a generalist can score it.

SDE-2 CORE CHAPTER

Chapter 5 - Conflict, Disagreement, and Influence

Learning objectives

By the end of this chapter, you should be able to:

- choose a conflict story with real stakes and a relationship that survived;
- show how you disagreed: with evidence, directly, and with respect;
- explain what you did when the decision went against you;
- show influence over people you did not manage; and
- avoid the conflict stories that quietly disqualify candidates.

WHY IT WORKS | Why this matters at SDE-2

"Tell me about a disagreement with a colleague" is one of the most common behavioral questions at every level, and one of the easiest to fail. At SDE-2 the bar rises: interviewers expect disagreements about technical direction or priorities, often with more senior people or other teams, resolved through evidence rather than escalation. They also expect you to be able to lose gracefully - disagree, then commit fully to the decision.

WHY IT WORKS | First-principles model

A conflict story is scored on three things:

- 1 **The disagreement was about substance** - a design, a priority, a risk - not personalities.
- 2 **Your conduct was effective and respectful**: you understood the other view, brought evidence, and spoke directly to the person rather than around them.
- 3 **The outcome and the relationship**: whether you won or lost, the decision improved or was committed to, and you still work well with the person.

Influence is the same skill without the conflict: changing a decision that you did not own.

Core terminology

- **Disagree and commit:** argue your position fully, then fully support the decision once it is made.
- **Steelman:** state the other person's position in its strongest form.
- **Escalation:** involving someone with more authority to resolve a disagreement.
- **Influence without authority:** changing others' decisions through evidence, relationships, and framing.
- **Pre-wiring:** discussing a proposal with key people individually before a group decision.

Detailed mechanics

Choosing the story

Good conflict stories involve a real technical or priority disagreement with real stakes, with someone whose opinion mattered - a senior engineer, a tech lead, another team, a product manager. The sample bank's S3 is a disagreement with the tech lead about a caching design.

Avoid:

- **Interpersonal drama** - "a colleague who was difficult". It invites the question of your contribution to the difficulty.
- **Conflicts where the other person was simply wrong** and you were simply right. Real disagreements have merit on both sides; a story without that reads as low empathy.
- **Stories that end in escalation as the first move.** Escalation is sometimes correct, but as the second step, after a direct conversation.

The shape of a good conflict story

TEXT

stakes	what decision, and why it mattered
their view	the other person's position, stated fairly (steelman)
your view	your position and the evidence behind it
conduct	how you raised it: directly, privately first, with data
resolution	what was decided and how
aftermath	the relationship, and what you learned

S3, told in that shape: the tech lead wanted a write-through cache for product data to cut read latency; the candidate worried about stale prices during flash sales. Their view was sound - read latency was the top complaint. The candidate measured how often prices changed during sales and found bursts of hundreds of updates per minute, then proposed a shorter TTL with explicit invalidation on price changes. They raised it one to one before the design review, not in it. The lead agreed to the invalidation but kept a longer TTL for non-price fields - a better design than either original. They went on to co-write the next design.

When you lose

Some of the strongest conflict stories are ones where the decision went the other way. The evidence to include:

- you argued your case fully, with data;
- once it was decided, you committed: implemented the decision well, and did not relitigate it in every meeting;
- if your concern later proved right, you raised it with evidence and without "I told you so" - and if it proved wrong, you said so.

This is what "disagree and commit" means, and it is scored directly at Amazon under Have Backbone; Disagree and Commit, and indirectly almost everywhere else.

Influence without authority

Influence stories - S5, the cross-team library - follow a recognisable pattern:

- 1 **Find the other side's incentive.** What do the other teams care about? Fewer incidents, less on-call load, less code to maintain.
- 2 **Lower the cost of saying yes.** A working library, a migration guide, an offer to pair on the first integration.
- 3 **Show evidence from a pilot.** One team's results persuade the next.
- 4 **Pre-wire.** Talk to each lead individually before asking for a group decision.

The candidate who says "I convinced them because my idea was better" has told a weaker story than the one who says "I found out what each team was worried about and made adoption cheaper than not adopting".

With a manager

"Tell me about a time you disagreed with your manager" needs the same shape and extra care. Show that you raised it privately, with evidence, and accepted their authority to decide - and that it was about the work, not about them.

WATCH OUT | Failure modes and common mistakes

- **A story with no real disagreement** - "we discussed and agreed".
- **Villain stories** in which the other person was unreasonable and you were right.
- **Escalating first.** Direct conversation comes first.
- **Winning at the cost of the relationship.** The aftermath is part of the score.
- **Never losing.** A candidate who always won every disagreement is not credible.
- **Relitigating after the decision.** The opposite of commit.

TRY IT | Interview questions and model answers

Q: Tell me about a time you disagreed with a technical decision. A: S3: the caching design. State the lead's view fairly, the data you gathered on price-change bursts, that you raised it one to one before the review, the combined design that resulted, and that you co-wrote the next design together.

Q: Tell me about a time a decision went against you. A: A story where you argued with data, lost, committed, and implemented the decision well - and what happened next, including whether your concern turned out to be right.

Q: How do you get another team to do something they don't own? A: Find their incentive, lower the cost of saying yes, show a pilot's results, and talk to each lead before the group decision. S5 is the example: a working library, a migration guide, and one team's results.

TRY IT | Exercises

- 1 Write your main conflict story in the six-part shape above. State the other person's view in a way they would agree with.
- 2 Find a story where you lost a disagreement and committed. If you have none, look again - you almost certainly do.
- 3 For an influence story, write down the other team's incentive in one sentence.

RECAP | Chapter summary

Conflict stories are scored on substance, conduct, and aftermath. Choose a real disagreement about the work, state the other view fairly, show direct and evidence-based conduct, and describe a relationship that survived. Have a story where you lost and committed. Influence stories show that you understood the other side's incentives and made agreement cheap.

TRY IT | Revision checklist

- ☐ I have a conflict story about substance, with a fair statement of the other view.
- ☐ I raised the disagreement directly before escalating.
- ☐ I have a "disagree and commit" story where I lost.
- ☐ My influence story names the other side's incentive and a pilot.

SDE-2 CORE CHAPTER

Chapter 6 - Failure, Feedback, and Growth

Learning objectives

By the end of this chapter, you should be able to:

- choose a failure that was real, yours, and consequential;
- tell it in a way that shows ownership, recovery, and a lasting change;
- describe receiving hard feedback and what you did differently afterwards;
- describe giving hard feedback kindly and effectively; and
- answer "what is your biggest weakness?" without a disguised strength.

WHY IT WORKS | Why this matters at SDE-2

Failure and feedback questions test self-awareness, which interviewers treat as a predictor of growth. They are also where rehearsed candidates are most transparent: a "failure" that was really someone else's fault, or a weakness that is secretly a strength, tells the interviewer more than a real mistake would. At SDE-2, interviewers expect failures with real consequences - an incident, a missed commitment, a bad design decision - and a change in behaviour that lasted.

WHY IT WORKS | First-principles model

A failure story is scored almost entirely on what happened **after** the failure:

- 1 **Ownership:** you say plainly what you did wrong, without shifting blame.
- 2 **Recovery:** what you did to limit the damage.
- 3 **Prevention:** what you changed so that it would not happen again - to the system, the process, or your own habits.
- 4 **Evidence of the change:** a later situation where you behaved differently.

The failure itself only needs to be real and to matter.

Core terminology

- **Blameless post-mortem:** an incident review focused on systemic causes rather than individual fault.
- **Contributing factor:** a condition that made a failure possible or worse.
- **Corrective action:** a concrete change made to prevent recurrence.
- **Feedback:** specific, actionable information about behaviour and its effect.

Detailed mechanics

Choosing a failure

Good choices are real mistakes with consequences, where you were a primary cause:

- a change you shipped that caused an incident (S4 in the sample bank: a retry change that sent duplicate emails);
- a design decision that did not hold up and had to be reversed;
- an estimate or commitment you missed, and how you handled it;
- a problem you saw coming and did not raise early enough.

Avoid:

- failures that were someone else's fault in disguise;
- trivial failures ("I once forgot to update a README");
- catastrophic failures with no recovery or learning;
- failures from so long ago that the learning is not current.

The shape of a failure story

S4, told well:

What happened. I changed our email service's retry policy to handle a provider's intermittent timeouts. I retried on timeout without checking whether the provider had actually accepted the request. During a provider slowdown, about four thousand customers received the same order confirmation two or three times.

My part. The mistake was mine: I treated a timeout as a failure, when it only meant the outcome was unknown. The reviewer did not catch it either, but I wrote the change and did not test the timeout path.

Recovery. I rolled the change back within forty minutes, worked with support on a short apology email, and wrote the incident review.

Prevention. I added an idempotency key to every provider call so that a retry could not duplicate a send, and a test that simulates a timeout after the provider accepts. I also proposed a checklist item for any change to retry logic: "what happens if the first attempt actually succeeded?"

Lasting change. Six months later, reviewing a payments retry change, I asked exactly that question and found the same bug before it shipped. That idea later became the shared idempotency library.

The last paragraph is what distinguishes a strong answer: evidence that the learning changed later behaviour.

Receiving feedback

"Tell me about a time you received critical feedback" is scored on your response, not the feedback:

- **The feedback, stated specifically and fairly.** S8: "My design documents were long and buried the decision; reviewers could not tell what I was asking them to approve."
- **Your first reaction, honestly.** It is fine to say it stung.
- **What you did.** Asked for examples, rewrote the next document with the decision and alternatives on the first page, and asked the same reviewer to check it.
- **The result.** Reviews got faster; the one-page format became the team template.

Giving feedback

Giving hard feedback well is an SDE-2 signal, especially for mentoring and team health:

- give it **privately and promptly**, about **specific behaviour and its effect**, not character;
- **ask for their view** before concluding;
- **agree a next step**, and follow up.

"I told them their pull requests were too large" is weak. "I showed them two recent pull requests that had waited three days for review, explained that reviewers were skipping them because of size, and paired on splitting the next one; their review time dropped to under a day" is strong.

"What is your biggest weakness?"

Choose a real, work-relevant weakness that is not disqualifying for the role, and show what you are doing about it, with evidence:

- "I used to go deep on a problem before checking whether others had already solved it. I now spend the first hour of any new problem searching our design docs and asking in the team channel; twice in the last quarter that found an existing solution."

Avoid disguised strengths ("I work too hard") and weaknesses central to the role ("I find debugging tedious").

WATCH OUT | Failure modes and common mistakes

- **Blame-shifting**, even subtly: "the requirements were unclear", "QA missed it".
- **No prevention**, only recovery.
- **No evidence the learning lasted**.
- **Disguised-strength weaknesses**. Interviewers have heard them all.
- **Defensive retelling of feedback**. The score is about your response, not whether the feedback was fair.

TRY IT | Interview questions and model answers

Q: Tell me about a time you failed. A: S4: the retry change that duplicated emails. What I did wrong, stated plainly; rollback in forty minutes; idempotency keys and a timeout test as prevention; and six months later catching the same bug in a payments review.

Q: Tell me about critical feedback you received. A: S8: design documents that buried the decision. What the feedback was, what I did, and that the one-page format became the team template.

Q: What is your biggest weakness? A: A real, relevant weakness, the specific change I made, and evidence that the change is working.

TRY IT | Exercises

- 1 Write your main failure story in five parts: what happened, your part, recovery, prevention, lasting change.
- 2 If the "lasting change" part is empty, choose a different failure or find the later evidence.
- 3 Write down the hardest feedback you have received in two years, and what you changed afterwards.
- 4 Write an answer to "biggest weakness" that includes evidence of improvement.

RECAP | Chapter summary

Failure and feedback stories are scored on what you did afterwards. Choose a real, consequential mistake that was yours; state your part plainly; describe recovery, prevention, and evidence that the learning lasted. For feedback, the score is your response. For weaknesses, choose a real one and show evidence that you are improving.

TRY IT | Revision checklist

- ☐ My failure story is real, mine, and consequential.
- ☐ It includes prevention and a later example of changed behaviour.
- ☐ I have a received-feedback story and a given-feedback story.
- ☐ My weakness answer is real, relevant, and shows improvement.

SDE-2 CORE CHAPTER

Chapter 7 - Mentoring and Leadership Without Authority

Learning objectives

By the end of this chapter, you should be able to:

- show leadership as an individual contributor, without a management title;
- tell a mentoring story whose outcome is the other person's growth, not your help;
- describe raising the bar for a team through reviews, documents, or process;
- describe leading during an incident; and
- calibrate these stories to SDE-2 rather than to a staff or management role.

WHY IT WORKS | Why this matters at SDE-2

At SDE-1, leadership is a bonus. At SDE-2 it is expected in a specific form: an engineer who makes the people and systems around them better without being told to. Interviewers look for mentoring, technical leadership on a project, improvements to how the team works, and composure during incidents. Candidates with no management experience often assume they have nothing to say. Almost every SDE-2 candidate has these stories; they have just not framed them.

WHY IT WORKS | First-principles model

Leadership without authority is **changing outcomes through other people when you cannot instruct them**. The evidence is always the same shape: a situation where the team was worse off than it could be, what you did to change how others worked, and the lasting effect.

For mentoring specifically, the score is on the mentee's growth. "I answered their questions" is help; "they now answer other people's questions" is mentoring.

Core terminology

- **Technical lead (informal):** the engineer who drives a project's design and sequencing without managing its people.
- **Raising the bar:** improving a team's standards - reviews, testing, documentation, on-call.
- **Force multiplier:** work that makes several other people more effective.
- **Incident commander:** the person coordinating an incident response, distinct from those fixing it.

Detailed mechanics

Mentoring

The sample bank's S6: a new engineer's first on-call rotation.

- **Starting point:** they were anxious, had not seen the systems under load, and the previous new hire had escalated nearly every page.
- **What you did:** shadowed them for the first two shifts, then reversed roles and let them lead with you shadowing; walked through three past incidents using the runbooks; agreed a rule that they would spend fifteen minutes on a page before escalating.
- **Their growth:** by the fourth week they handled pages alone and escalated one in ten; six months later they were mentoring the next hire the same way.
- **What you learned:** you had been solving problems for mentees too quickly; now you ask what they would try first.

The last two bullets make it a mentoring story rather than a helping story.

Technical leadership on a project

SDE-2 candidates often led a project's technical direction without a title. Evidence to include:

- **you set the direction:** wrote the design, sequenced the work, identified the risks;
- **you enabled others:** split work so that others could own pieces, reviewed their designs, unblocked them;
- **you kept it on track:** noticed slippage early and changed the plan, not just the dates.

The database migration (S1) is also a technical-leadership story if told from this angle: the four-step plan let two teammates own the backfill and the read switch-over in parallel.

Raising the bar

Improvements to how a team works are strong SDE-2 evidence because they outlast you:

- a design-document template that made reviews faster (S8's outcome);
- a checklist for risky changes, such as the retry-logic question from S4;
- a runbook standard that let new on-call engineers handle incidents alone;
- a code-review norm - response within one working day, or size limits - that cut cycle time.

State the before and after with numbers where you can: review time, incident count, on-call escalations.

Leading during an incident

Incident leadership is scored on composure and coordination, not heroics:

- **you separated coordination from fixing:** someone led communication while others investigated;
- **you prioritised mitigation over diagnosis:** rolled back or failed over first, found the root cause afterwards;
- **you communicated on a schedule:** status every fifteen or thirty minutes, even without news;
- **you owned the follow-up:** a blameless post-mortem with corrective actions that were actually completed.

S2, the payment outage, belongs here: the candidate recognised that three engineers were all debugging and nobody was updating support, took the coordinator role, asked one engineer to prepare a rollback while the others kept investigating, and rolled back at the twenty-minute mark when the cause was still unclear.

Calibrating to SDE-2

Do not over-reach. An SDE-2 candidate who describes managing people, setting organisational strategy, or hiring decisions may be asked why they are interviewing for SDE-2. Aim for team-level leadership: your project, your team's practices, your mentees, your incidents.

WATCH OUT | Failure modes and common mistakes

- **"I don't have leadership experience."** You almost certainly do; frame it.
- **Helping presented as mentoring.** Show the mentee's growth.
- **Heroic incident stories.** Coordination and mitigation, not all-night debugging.
- **Improvements without a measured effect.**
- **Over-reaching** to manager or staff scope.

TRY IT | Interview questions and model answers

Q: Tell me about a time you mentored someone. A: S6: the new engineer's first on-call. Their starting point, the shadow-then-reverse approach, the escalation rule, and the result - handling pages alone within four weeks and mentoring the next hire six months later.

Q: Tell me about a time you led without formal authority. A: The migration as a technical-leadership story: I wrote the design and the four-step plan, split the work so that two teammates could own the backfill and the read switch-over, and reviewed their designs.

Q: Tell me about a time you improved how your team works. A: The one-page design-document format from S8: review turnaround dropped from about a week to two days, and the team adopted it as the template.

Q: Walk me through how you handled a production incident. A: S2: took coordination when nobody was communicating, rolled back at twenty minutes before finding the cause, sent updates every fifteen minutes, and completed the post-mortem actions.

TRY IT | Exercises

- 1 Write a mentoring story in which the final sentence is about the mentee, not you.
- 2 List three things you changed about how your team works. Put a before-and-after number on one.
- 3 Write your incident story with one sentence each for coordination, mitigation, communication, and follow-up.

RECAP | Chapter summary

Leadership at SDE-2 means improving outcomes through other people without authority: mentoring measured by the mentee's growth, technical leadership that enabled others, lasting improvements to how the team works, and calm coordination during incidents. Frame the stories you already have, measure their effect, and stay at team scope.

TRY IT | Revision checklist

- ☐ My mentoring story ends with the mentee's growth.
- ☐ I have a technical-leadership story that shows how I enabled others.
- ☐ I have a team-improvement story with a measured effect.
- ☐ My incident story emphasises coordination and mitigation.

SDE-2 CORE CHAPTER

Chapter 8 - Company Formats and Leadership Principles

Learning objectives

By the end of this chapter, you should be able to:

- map your story bank onto a company's published values, using Amazon's Leadership Principles as the worked example;
- find the principles your bank does not yet cover;
- adapt the same stories to rounds that are labelled differently at different companies; and
- use your recruiter's preparation material as the specification.

WHY IT WORKS | Why this matters at SDE-2

The signals in this book are universal, but companies name and weight them differently, and some structure whole rounds around their own values. A candidate who walks into an Amazon loop without having mapped their stories to the Leadership Principles, or into any loop without having read the recruiter's preparation guide, gives away evidence they already have.

WHY IT WORKS | First-principles model

Every company's behavioral rubric is a **relabelling and reweighting of roughly the same signals**. Preparation for a specific company is therefore not a new story bank; it is a mapping from your existing stories onto the company's names, followed by a check for gaps.

Specification boundary: interview formats change, and companies describe them in their own preparation material. Treat that material as the specification, and treat anything in this chapter - other than Amazon's published list of principles - as a general pattern to verify, not a guarantee of a particular company's current format.

Core terminology

- **Leadership Principles (LPs):** Amazon's published list of sixteen principles used across its hiring process.
- **Values interview:** a round explicitly scored against a company's stated values.
- **Hiring-manager round:** an interview with the prospective manager, often mixing behavioral and technical discussion.
- **Recruiter preparation guide:** the material a recruiter sends describing the loop and what each round evaluates.

Detailed mechanics

Amazon's Leadership Principles

Amazon publishes sixteen Leadership Principles and uses them throughout its interviews; behavioral questions in an Amazon loop are typically each aimed at one or two of them. The list, as published:

TEXT	
Customer Obsession	Think Big
Ownership	Bias for Action
Invent and Simplify	Frugality
Are Right, A Lot	Earn Trust
Learn and Be Curious	Dive Deep
Hire and Develop the Best	Have Backbone; Disagree and Commit
Insist on the Highest Standards	Deliver Results
Strive to be Earth's Best Employer	Success and Scale Bring Broad Responsibility

The companion maps each of the twelve signals from chapter 1 onto the principles it can evidence - a preparation aid, not an official Amazon rubric - and counts how many stories in the sample bank reach each one:

TEXT	
Customer Obsession	1
Ownership	3
Invent and Simplify	2
Are Right, A Lot	2
Learn and Be Curious	3
Hire and Develop the Best	1
Insist on the Highest Standards	3
Think Big	2
Bias for Action	5
Frugality	2
Earn Trust	4
Dive Deep	4
Have Backbone; Disagree and Commit	1
Deliver Results	4
Strive to be Earth's Best Employer	0
Success and Scale Bring Broad Responsibility	0
no story yet for: [Strive to be Earth's Best Employer, Success and Scale Bring Broad Responsibility]	

Two findings follow. First, the two newest principles - added in 2021 and about people and broader impact - have no story at all in a bank built around the generic signals. Stories that fit them: making on-call sustainable for a team, improving inclusion in reviews or hiring, or considering the wider effect of a system on users and the business - accessibility, privacy, or reliability for vulnerable users. Second, Customer Obsession, Hire and

Develop the Best, and Have Backbone rest on a single story each, which is the same gap as the single-story conflict signal in chapter 2, seen through a different lens.

Using the mapping

For an Amazon loop:

- 1 Map every story in your bank onto its principles, as above.
- 2 Make sure every principle has at least one story and the most common ones - Customer Obsession, Ownership, Dive Deep, Deliver Results, Have Backbone - have two.
- 3 Use the principle's language naturally in your answer where it is honest: "I wanted to dive deep here because the averages hid the problem." Do not recite principle names; interviewers notice.

Other common formats

Formats vary, but three patterns recur across large companies:

- **A dedicated values or behavioral round**, scored on a published or semi-published set of values. Map your bank onto the named values exactly as with Amazon's principles.
- **A hiring-manager round** mixing behavioral questions with a discussion of a past project in technical depth. Your anchor story (chapter 2) is the natural centrepiece; expect deep technical probes on it.
- **Behavioral questions embedded in technical rounds** - "tell me about a system you designed" at the start of a design round. Keep these short, so that the technical part has its time.

For any company, the recruiter's preparation guide is the best available description of the current loop. Read it, note the values or signals it names, and map your bank onto them.

The same story, three labels

S3, the caching disagreement, is:

- **Have Backbone; Disagree and Commit** at Amazon;
- **conflict resolution** or **collaboration** in a generic behavioral rubric;
- a **technical judgement** example in a hiring-manager conversation.

The story does not change; the emphasis does. For Have Backbone, dwell on raising the concern with evidence to a more senior person. For collaboration, dwell on the combined design. For technical judgement, dwell on the price-change data.

WATCH OUT | Failure modes and common mistakes

- **Not reading the recruiter's guide.** It is the closest thing to a specification.
- **A new story bank per company.** Map the existing one instead.
- **Reciting principle names.** Use the ideas, not the labels.
- **Ignoring the newer principles** in an Amazon loop.
- **Long behavioral answers inside technical rounds.** Keep them short.

TRY IT | Interview questions and model answers

Q: Tell me about a time you insisted on high standards. (*Amazon: Insist on the Highest Standards*) A: S1's backfill: stopping a migration on a 2% data mismatch rather than continuing to the deadline, fixing the root cause, and re-running the comparison.

Q: Tell me about a time you went deep into data to solve a problem. (*Dive Deep*) A: S3: measuring price-change bursts during sales to show where the proposed cache would serve stale prices.

Q: Tell me about a time you made things better for the people on your team. (*Strive to be Earth's Best Employer*) A: The gap the companion found. A good fit is S6 reframed: redesigning the on-call onboarding so that new engineers were not overwhelmed, with its effect on escalations and on the next hire.

TRY IT | Exercises

- 1 Map your own bank onto Amazon's sixteen principles. Which have no story, and which have only one?
- 2 Take the values list from any company you are interviewing with and map your bank onto it.
- 3 Tell S3, or your own conflict story, three ways: emphasising backbone, collaboration, and technical judgement.

RECAP | Chapter summary

Company rubrics relabel and reweight the same core signals, so prepare one story bank and map it onto each company's values. Amazon's sixteen Leadership Principles are the clearest published example; the sample bank covered fourteen of them and missed the two newest. Treat the recruiter's preparation guide as the specification, and adjust emphasis - not stories - to the label a question is scored against.

TRY IT | Revision checklist

- ☐ I can list Amazon's sixteen Leadership Principles.
- ☐ My bank maps onto every principle, with two stories for the most common ones.
- ☐ I have read the preparation guide for each company I am interviewing with.
- ☐ I can tell one story with three different emphases.

SDE-2 CORE CHAPTER

Chapter 9 - The Hiring-Manager Round and Closing Well

Learning objectives

By the end of this chapter, you should be able to:

- tell your career story in two minutes, with a clear direction;
- answer "why this company?" and "why are you leaving?" honestly and positively;
- present a past project in depth for a hiring manager;
- ask questions that are useful to you and informative to the interviewer; and
- recognise and recover from the red flags that end otherwise good rounds.

WHY IT WORKS | Why this matters at SDE-2

The hiring-manager round is where the question "do I want this person on my team?" is answered most directly. It blends behavioral questions with a project deep dive and a conversation about motivation. Candidates prepare the stories and forget the rest: the opening "tell me about yourself", the "why here?", and the questions they ask at the end - all of which the manager is also scoring.

WHY IT WORKS | First-principles model

The hiring manager is assessing three things beyond the behavioral signals:

- 1 **Trajectory:** are you growing, and in a direction this team needs?
- 2 **Motivation:** do you want this role specifically, for reasons that will survive the first hard month?
- 3 **Team fit in the working sense:** how you would collaborate with this team on this kind of work - not personality.

Everything in this chapter serves one of those three.

Core terminology

- **Career narrative:** a short account of your path that explains your choices and points forward.
- **Project deep dive:** a detailed discussion of one past project, usually technical and behavioral at once.
- **Reverse interview:** the questions you ask the interviewer.
- **Red flag:** a behaviour that outweighs otherwise good evidence.

Detailed mechanics

"Tell me about yourself"

Two minutes, in three parts:

- 1 **Now:** your current role and scope in one or two sentences, with one concrete achievement.
- 2 **How you got here:** two or three choices, each with the reason - what you were looking for, and what you learned.
- 3 **Why this role:** where you are heading and how this role fits.

A backend engineer's version: "I'm a backend engineer at a payments company, where I own the order and checkout services - most recently I moved order data onto its own database without downtime. I started in a services consultancy, moved to product engineering to own systems long term, and have spent the last three years on distributed systems and reliability. I'm looking for a team where I can take on larger system ownership, which is why this platform role stood out."

It is not a resume walk-through. It is the story that explains the resume.

"Why this company?" and "Why are you leaving?"

- **Why here:** specific to the team and its problems, not the brand. Read the job description, the engineering blog, and anything public about the team's systems, and connect them to your experience and your direction.
- **Why leaving:** forward-looking and honest. "I've learned a lot owning these services; I'm looking for a larger scope in distributed systems than my current team has" works. Criticising your current employer does not, even when justified.

The project deep dive

Many hiring managers ask you to walk through one project in depth. Use your anchor story:

- **the problem and why it mattered**, in business terms;
- **the architecture**, drawn if possible, with your part marked;
- **the key decisions**, each with the alternatives you rejected;
- **what went wrong**, and what you did about it;
- **the result and what you would change.**

Expect technical probes. This is where a story you truly lived is indispensable.

Your questions

Your questions are scored, and they are also your only chance to learn what the job is really like. Good questions are specific and show judgement:

- "What does a successful first six months look like for this role?"
- "What is the biggest technical risk the team is carrying right now?"
- "How does the team handle on-call, and what was the last significant incident?"
- "How are technical decisions made when engineers disagree?"
- "What does growth to the next level look like here, and who recently made it?"

Avoid questions answered on the careers page, and save compensation for the recruiter unless the manager raises it.

Red flags and recovery

Behaviours that can outweigh good evidence:

- **Blaming others** for failures, in any story.
- **Disparaging a current or former employer.**
- **Inconsistency** between stories, or between a story and the resume.
- **Claiming credit for team work** that probes reveal was not yours.
- **No questions** at the end, which reads as low interest.

Recovery is possible within the round. If you realise you over-claimed, correct it: "I should be clearer - the design was mine, but the implementation was shared with two teammates." Correcting yourself is evidence of integrity, which the interviewer is also scoring.

After the loop

Send a short thank-you note to the recruiter if that is the norm at the company. Write down, the same day, every question you were asked and how you answered; if you interview elsewhere, that is your best preparation material. If the outcome is negative, ask the recruiter for any feedback they can share.

WATCH OUT | Failure modes and common mistakes

- **A resume walk-through** instead of a narrative.
- **Generic motivation:** "it's a great company."
- **Negative reasons for leaving.**
- **A deep dive without trade-offs** or without your part marked.
- **Weak or no questions.**

TRY IT | Interview questions and model answers

Q: Tell me about yourself. A: Now, how you got here, why this role - two minutes, one concrete achievement, and a direction that this role fits.

Q: Why do you want to leave your current job? A: A forward-looking reason about the scope or kind of work you want next, stated without criticising your employer.

Q: Walk me through a project you're proud of. A: The anchor story as a deep dive: problem, architecture with your part marked, decisions with alternatives, what went wrong, result, and what you would change.

Q: Do you have any questions for me? A: Two or three specific questions - about the first six months, the team's biggest technical risk, and how disagreements are resolved.

TRY IT | Exercises

- 1 Write and time your two-minute "tell me about yourself".
- 2 Write "why this company" for one real company you are targeting, naming the team's specific problems.
- 3 Draw the architecture of your anchor project and mark your part.
- 4 Write five questions to ask, and cross out any whose answer is on the company's website.

RECAP | Chapter summary

The hiring-manager round scores trajectory, motivation, and working fit alongside the behavioral signals. Prepare a two-minute narrative, specific reasons for this role, positive reasons for leaving, a project deep dive with your part and your decisions marked, and questions that show judgement. Avoid blame and inconsistency, and correct yourself openly if you over-claim.

TRY IT | Revision checklist

- ☐ My "tell me about yourself" is two minutes and points forward.
- ☐ My "why here" names the team's specific problems.
- ☐ My anchor project is ready as a deep dive with decisions and alternatives.
- ☐ I have five questions that are not answered on the careers page.

SDE-2 CORE CHAPTER

Chapter 10 - Behavioral Interview Drills

Behavioral preparation is only real when it is written down and spoken aloud. Each drill names the artifact to produce. Do them with your own stories; the worked solutions use the sample bank from the chapters to show what a strong artifact looks like.

Foundation: the bank

D1. Audit a story bank. Using the companion's coverage table for the sample bank (chapter 2), *produce*: the signals covered by only one story, the question type most at risk in a real loop, and one concrete story idea to fill each gap for a backend engineer.

D2. Plan a loop. An Amazon loop is expected to ask about Customer Obsession, Ownership, Dive Deep, Have Backbone, Deliver Results, and Earn Trust. *Produce*: a story assignment from the sample bank with no story used twice, noting which assignments are weakest.

D3. Upgrade a weak answer. Weak answer: "We had a lot of incidents with the payment service, so our team decided to refactor it. It took a few months but in the end things got a lot better and everyone was happy." *Produce*: a list of what is missing, and the five questions you would ask the candidate to recover the evidence.

Structure and delivery

D4. Rebalance an answer. A draft answer has 120 words of situation, 40 of task, 90 of action, and 30 of result. *Produce*: the section shares, the estimated speaking time at 150 words per minute, and a target word budget for each section that meets the chapter 3 targets at the same total length.

D5. Fix "we". Rewrite this action section so that the candidate's own decisions are clear without taking credit for the team's work: "We looked at the logs and we found the problem was in the retry logic. We decided to add idempotency keys and we rolled it out over two weeks."

Signal-specific stories

D6. Conflict you lost. *Produce*: the six-part outline (stakes, their view, your view, conduct, resolution, aftermath) of a disagreement where the decision went against you and you committed.

D7. A real failure. *Produce*: a five-part failure story (what happened, your part, recovery, prevention, lasting change), and check that the last part contains a later, specific example.

D8. Mentoring measured by the mentee. *Produce*: a mentoring story whose final two sentences describe what the mentee could do afterwards that they could not do before.

Company formats

D9. Cover the newest principles. *Produce:* one story idea for each of Amazon's two newest Leadership Principles - Strive to be Earth's Best Employer and Success and Scale Bring Broad Responsibility - that a backend engineer at SDE-2 level could plausibly have.

Challenge

D10. Full mock behavioral round. Have someone ask you six questions drawn from different signals, with at least two probes per answer. Record it. *Produce:* for each answer, its length, its action share, its "I"/"we" balance, and whether each probe found real detail - then list the two stories to rework first.

SDE-2 CORE CHAPTER

Chapter 11 - Behavioral Interview Drills: Worked Solutions

These solutions use the sample story bank from the chapters. Your artifacts should use your own stories; what transfers is the shape and the level of detail.

Foundation: the bank

D1. Audit a story bank

The companion's coverage table shows four signals resting on a single story: **conflict** (S3), **mentoring** (S6), **customer focus** (S7), and **prioritization** (S7).

The most at-risk question type is **conflict**, because it is among the most frequently asked and a loop may ask it twice - once directly and once as "tell me about a time you disagreed with your manager". With one story, the second question has no fresh answer.

Gap-filling ideas for a backend engineer:

- **Conflict:** a disagreement with a product manager about shipping a feature without an alert or dashboard, resolved by agreeing a minimum observability bar.
- **Mentoring:** reviewing an intern's first design document and coaching them through the review meeting.
- **Customer focus:** pushing back on an API change after reading support tickets that showed how merchants actually used the field.
- **Prioritization:** choosing to fix a flaky test suite before starting a planned feature, and the argument you made for it.

D2. Plan a loop

TEXT	
Customer Obsession	-> S7 (scope cut kept what the regulator and users needed)
Ownership	-> S4 (the retry bug, owned end to end)
Dive Deep	-> S2 (payment outage investigation)
Have Backbone; Disagree and Commit	-> S3 (caching disagreement)
Deliver Results	-> S1 (migration, two weeks early)
Earn Trust	-> S5 (three-team library adoption)

No story is used twice. The weakest assignments are **Ownership via S4**, because a failure story is an indirect ownership story and S1 - the strongest ownership story - is already spent on Deliver Results; and **Customer Obsession via S7**, the bank's only customer story. Both point back to the gaps in D1.

D3. Upgrade a weak answer

What is missing: the candidate's own role ("our team decided"), any decision or alternative, any number ("a lot of incidents", "a lot better"), any difficulty, and any learning. There is nothing to score.

Five questions that recover the evidence:

- 1 How many incidents, over what period, and what did they have in common?
- 2 What was your specific role - did you propose the refactor, design it, or implement part of it?
- 3 What alternatives did you consider, and why did you reject them?
- 4 What went wrong or was hard during the refactor, and what did you do about it?
- 5 What was the incident rate afterwards, and how did you measure it?

Structure and delivery

D4. Rebalance an answer

TEXT

```
current:  situation 120 (42.9%)  task 40 (14.3%)  action 90 (32.1%)  result 30 (10.7%)
total      280 words  ->  112 s at 150 words per minute
```

Setup is 57% of the answer and action only 32% - the answer spends almost two thirds of its time before the candidate does anything. At the same 280 words, a budget that meets the targets (action at least 55%, situation and task at most 25%):

TEXT

```
target:  situation 40    task 25    action 160   result 55    total 280
shares   14.3%          8.9%      57.1%      19.6%
```

The 80 words cut from the situation become decisions, alternatives, and numbers in the action section.

D5. Fix "we"

I read the logs from the three failed payments and traced all of them to the retry logic: we retried on timeout without knowing whether the first attempt had succeeded. I proposed idempotency keys on every provider call and wrote the change; a teammate reviewed it and built the test harness for the timeout case. We rolled it out to one region first and then the rest over two weeks.

"I" marks the candidate's decisions and work; "we" marks the shared rollout; the teammate's contribution is credited precisely.

Signal-specific stories

D6. Conflict you lost

TEXT

stakes	whether to split a monolith's reporting module into its own service this quarter
their view	the lead wanted it now: reporting queries were slowing the main database
your view	add a read replica first; the split would take a quarter and the replica a week
conduct	raised it one to one with a cost comparison, then in the design review
resolution	the lead chose the split, because reporting would grow and a replica only deferred it
aftermath	I committed: I took the hardest part, the data-sync design, and it shipped on time; a year later reporting load had tripled, and the lead had been right

The last line is what makes it strong: the candidate lost, committed fully, and says plainly that the other view was correct.

D7. A real failure

Use the five-part shape from chapter 6. The check is the fifth part: it must name a *later, specific* event where the lesson changed behaviour - "six months later, reviewing a payments retry change, I asked what happens if the first attempt succeeded and found the same bug". A general statement such as "since then I always test edge cases" is not evidence.

D8. Mentoring measured by the mentee

End with the mentee, for example: "Four weeks later she was taking on-call shifts alone and escalating about one page in ten. Six months later she onboarded the next new hire using the same shadow-then-reverse approach."

Company formats

D9. Cover the newest principles

- **Strive to be Earth's Best Employer:** reducing on-call load for your team - for example, fixing the top five noisy alerts so that out-of-hours pages dropped by half - or making code review more inclusive for new or remote teammates, with a measured effect.
- **Success and Scale Bring Broad Responsibility:** considering the wider effect of a system - adding rate limits and abuse controls to a public API before launch, or making a data pipeline delete personal data on request - and the trade-off it cost.

Challenge

D10. Full mock behavioral round

Score each answer against chapter 3's targets: one and a half to three minutes; action at least 55%; "I" dominant in the action section; every probe answered with a specific detail. Most first mocks show the same two problems: setup that runs too long, and results without numbers. Rework the two stories with the lowest scores first, and repeat the mock with a different interviewer.

SDE-2 CORE CHAPTER

Chapter 12 - Executable Story Bank Model

This dependency-free Java 21 model treats a story bank as data: it computes signal coverage and gaps, plans a loop without over-using a story, times a STAR answer by section, counts 'I' against 'we', and maps the bank onto Amazon's sixteen Leadership Principles.

JAVA (continued 1/11)

```
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Executable checks for behavioral-interview preparation.
 *
 * A story bank is data: stories tagged with the signals they demonstrate. That
 * makes coverage, reuse, and gaps computable instead of a feeling. A written
 * answer is also data: its sections can be timed and its pronouns counted.
 * Every number quoted in the chapters is printed here. Run it with:
 *
 *     javac --release 21 StoryBankModel.java && java -ea StoryBankModel
 *
 * Sections map to chapters:
 * 1. Signal coverage of a story bank, and its gaps           (ch 2)
 * 2. Choosing stories for a loop without over-reusing one   (ch 2)
 * 3. Timing a STAR answer by section                         (ch 3)
 * 4. The "I" versus "we" ratio of an answer                 (ch 3)
 * 5. Mapping the bank onto Amazon's Leadership Principles   (ch 8)
 */
public final class StoryBankModel {

    private StoryBankModel() {}

    static void check(boolean condition, String message) {
        if (!condition) {
```

JAVA (continued 2/11)

```
        throw new IllegalStateException("check failed: " + message);
    }
}

// -----
// Signals and a sample story bank
// -----

enum Signal {
    OWNERSHIP,
    AMBIGUITY,
    TECHNICAL_DEPTH,
    DELIVERY,
    CONFLICT,
    INFLUENCE,
    FAILURE,
    FEEDBACK,
    MENTORING,
    CUSTOMER_FOCUS,
    PRIORITIZATION,
    INCIDENT
}

record Story(String id, String title, Set<Signal> signals) {}

/** A composite bank for a backend engineer with about five years' experience. */
static List<Story> sampleBank() {
    return List.of(
        new Story(
            "S1",
            "Migrated order service off a shared database",
            EnumSet.of(
                Signal.OWNERSHIP,
                Signal.AMBIGUITY,
```

JAVA (continued 3/11)

```
        Signal.technicalDepth,
        Signal.Delivery)),
    new Story(
        "S2",
        "Led the payment outage response and the fix",
        EnumSet.of(Signal.Incident, Signal.Ownership, Signal.TechnicalDepth)),
    new Story(
        "S3",
        "Disagreed with the tech lead on a caching design",
        EnumSet.of(Signal.Conflict, Signal.TechnicalDepth, Signal.Influence)),
    new Story(
        "S4",
        "Shipped a retry change that caused duplicate emails",
        EnumSet.of(Signal.Failure, Signal.Ownership, Signal.Incident)),
    new Story(
        "S5",
        "Convinced three teams to adopt a shared idempotency library",
        EnumSet.of(Signal.Influence, Signal.Ambiguity)),
    new Story(
        "S6",
        "Mentored a new engineer through their first on-call",
        EnumSet.of(Signal.Mentoring, Signal.Feedback)),
    new Story(
        "S7",
        "Cut scope to hit a regulatory deadline",
        EnumSet.of(Signal.Prioritization, Signal.Delivery, Signal.CustomerFocus)),
    new Story(
        "S8",
        "Received blunt review feedback on a design doc",
        EnumSet.of(Signal.Feedback, Signal.Failure)));
}

static Map<Signal, List<String>> coverage(List<Story> bank) {
    Map<Signal, List<String>> map = new EnumMap<>(Signal.class);
```

JAVA (continued 4/11)

```
    for (Signal s : Signal.values()) {
        map.put(s, new ArrayList<>());
    }
    for (Story story : bank) {
        for (Signal s : story.signals()) {
            map.get(s).add(story.id());
        }
    }
    return map;
}

static void storyCoverage() {
    System.out.println("== 1. Signal coverage of an 8-story bank ==");
    Map<Signal, List<String>> map = coverage(sampleBank());
    List<Signal> thin = new ArrayList<>();
    for (Map.Entry<Signal, List<String>> e : map.entrySet()) {
        System.out.printf(
            " %-15s %d %s %s%n",
            e.getKey(),
            e.getValue().size(),
            e.getValue().size() == 1 ? "story " : "stories",
            e.getValue());
        if (e.getValue().size() < 2) {
            thin.add(e.getKey());
        }
    }
    System.out.printf(" signals with fewer than two stories: %s%n", thin);
    check(
        thin.equals(
            List.of(
                Signal.CONFLICT,
                Signal.MENTORING,
                Signal.CUSTOMER_FOCUS,
                Signal.PRIORITIZATION)),
```

JAVA (continued 5/11)

```

        "the sample bank is thin on conflict, mentoring, customer focus, prioritization");
    check(
        map.values().stream().noneMatch(List::isEmpty),
        "every signal has at least one story");
}

// -----
// 2. Choosing stories for a loop
// -----

/**
 * Assign one story per question, preferring the story that covers the
 * question's signal with the fewest other uses so far, and never using
 * one story more than maxUses times across the loop.
 */
static Map<String, String> assign(List<Story> bank, List<Signal> questions, int maxUses) {
    Map<String, Integer> uses = new LinkedHashMap<>();
    Map<String, String> plan = new LinkedHashMap<>();
    for (int q = 0; q < questions.size(); q++) {
        Signal wanted = questions.get(q);
        Story best = null;
        for (Story story : bank) {
            if (!story.signals().contains(wanted)
                || uses.getOrDefault(story.id(), 0) >= maxUses) {
                continue;
            }
            if (best == null
                || uses.getOrDefault(story.id(), 0) < uses.getOrDefault(best.id(), 0)
                || (uses.getOrDefault(story.id(), 0).equals(uses.getOrDefault(best.id(), 0))
                    && story.signals().size() < best.signals().size())) {
                best = story;
            }
        }
        String key = "Q" + (q + 1) + " " + wanted;
    }
}

```

JAVA (continued 6/11)

```

        if (best == null) {
            plan.put(key, "GAP");
        } else {
            plan.put(key, best.id());
            uses.merge(best.id(), 1, Integer::sum);
        }
    }
    return plan;
}

static void loopPlanning() {
    System.out.println();
    System.out.println("== 2. Planning a six-question loop, each story used at most once ==");
    List<Signal> loop =
        List.of(
            Signal.OWNERSHIP,
            Signal.CONFLICT,
            Signal.FAILURE,
            Signal.INCIDENT,
            Signal.INFLUENCE,
            Signal.OWNERSHIP);
    Map<String, String> plan = assign(sampleBank(), loop, 1);
    plan.forEach((q, s) -> System.out.printf(" %-18s -> %s\n", q, s));
    check(!plan.containsValue("GAP"), "the bank covers this loop with no story reused");

    List<Signal> harder = List.of(Signal.MENTORING, Signal.FEEDBACK, Signal.MENTORING);
    Map<String, String> strained = assign(sampleBank(), harder, 1);
    System.out.printf(" mentoring, feedback, mentoring again -> %s\n", strained.values());
    check(strained.containsValue("GAP"), "one mentoring story cannot answer two questions");
}

// -----
// 3 and 4. Timing and pronouns in a written answer
// -----

```

JAVA (continued 7/11)

```
static final String SITUATION =
    "Our checkout service and the order service shared one PostgreSQL database. "
    + "Every schema change needed both teams, and a slow order query had "
    + "caused two checkout incidents in one quarter.";

static final String TASK =
    "I owned the plan to give the order service its own database without "
    + "downtime, and I had one quarter to do it.";

static final String ACTION =
    "I started by mapping every query that crossed the boundary, and found "
    + "fourteen, three of them joins that checkout depended on. I proposed "
    + "replacing the joins with an order-summary API, and wrote a one-page "
    + "design that listed the three options I had rejected and why. The "
    + "checkout lead pushed back on latency, so I built a prototype and "
    + "measured it: the API added four milliseconds at p99, which we agreed "
    + "was acceptable. I then ran the migration in four steps, each "
    + "reversible: dual writes behind a flag, a backfill with row-count and "
    + "checksum comparison, reads switched one endpoint at a time, and "
    + "finally removing the old tables. During the backfill I found that "
    + "two percent of rows differed because of a timezone bug in an old "
    + "job, so I stopped, fixed the job, and re-ran the comparison before "
    + "continuing. I wrote a runbook for each step so that anyone on call "
    + "could roll back.";

static final String RESULT =
    "We finished two weeks early with no customer-facing downtime. Checkout "
    + "had no database-related incidents in the next two quarters, and "
    + "schema changes stopped needing cross-team review. I would now start "
    + "the data comparison earlier, because finding the timezone bug late "
    + "cost us a week.";

static int words(String text) {
    String trimmed = text.trim();
    return trimmed.isEmpty() ? 0 : trimmed.split("\\s+").length;
}
```

JAVA (continued 8/11)

```

static int count(Pattern pattern, String text) {
    Matcher m = pattern.matcher(text.toLowerCase(Locale.ROOT));
    int n = 0;
    while (m.find()) {
        n++;
    }
    return n;
}

static void answerTiming() {
    System.out.println();
    System.out.println("== 3. Timing a STAR answer at 150 words per minute ==");
    String[] names = {"Situation", "Task", "Action", "Result"};
    String[] parts = {SITUATION, TASK, ACTION, RESULT};
    int total = 0;
    for (String p : parts) {
        total += words(p);
    }
    for (int i = 0; i < parts.length; i++) {
        int w = words(parts[i]);
        System.out.printf(
            " %-9s %3d words  %4.1f%%  %3.0f s%n",
            names[i], w, 100.0 * w / total, w / 150.0 * 60);
    }
    double seconds = total / 150.0 * 60;
    System.out.printf(
        " total      %3d words          %3.0f s (%.1f min)%n",
        total, seconds, seconds / 60);
    double actionShare = (double) words(ACTION) / total;
    double setupShare = (double) (words(SITUATION) + words(TASK)) / total;
    check(actionShare >= 0.55, "action is at least 55% of the answer");
    check(setupShare <= 0.25, "situation and task together are at most 25%");
    check(seconds >= 90 && seconds <= 180, "the answer runs between 1.5 and 3 minutes");
}

```

JAVA (continued 9/11)

```
System.out.println();
System.out.println("== 4. \"I\" versus \"we\" ==");
Pattern singular = Pattern.compile("\\b(i|my|me)\\b");
Pattern plural = Pattern.compile("\\b(we|our|us)\\b");
String all = String.join(" ", parts);
String actionOnly = ACTION;
int iAll = count(singular, all);
int weAll = count(plural, all);
int iAction = count(singular, actionOnly);
int weAction = count(plural, actionOnly);
System.out.printf(" whole answer: %d singular, %d plural%n", iAll, weAll);
System.out.printf(" action section: %d singular, %d plural%n", iAction, weAction);
check(iAction > 3 * weAction, "the action section is about what the candidate did");
check(weAll > 0, "shared outcomes are still credited to the team");
}

// -----
// 5. Amazon Leadership Principles coverage
// -----

static final List<String> AMAZON_LPS =
    List.of(
        "Customer Obsession",
        "Ownership",
        "Invent and Simplify",
        "Are Right, A Lot",
        "Learn and Be Curious",
        "Hire and Develop the Best",
        "Insist on the Highest Standards",
        "Think Big",
        "Bias for Action",
        "Frugality",
        "Earn Trust",
```

JAVA (continued 10/11)

```

        "Dive Deep",
        "Have Backbone; Disagree and Commit",
        "Deliver Results",
        "Strive to be Earth's Best Employer",
        "Success and Scale Bring Broad Responsibility");

/** Which LPs a signal can evidence. A mapping, not an official Amazon rubric. */
static Map<Signal, List<String>> lpMapping() {
    Map<Signal, List<String>> m = new EnumMap<>(Signal.class);
    m.put(Signal.OWNERSHIP, List.of("Ownership", "Deliver Results"));
    m.put(Signal.AMBIGUITY, List.of("Bias for Action", "Think Big"));
    m.put(Signal.TECHNICAL_DEPTH, List.of("Dive Deep", "Insist on the Highest Standards"));
    m.put(Signal.DELIVERY, List.of("Deliver Results", "Frugality"));
    m.put(Signal.CONFLICT, List.of("Have Backbone; Disagree and Commit", "Earn Trust"));
    m.put(Signal.INFLUENCE, List.of("Earn Trust", "Invent and Simplify"));
    m.put(Signal.FAILURE, List.of("Learn and Be Curious", "Are Right, A Lot"));
    m.put(Signal.FEEDBACK, List.of("Earn Trust", "Learn and Be Curious"));
    m.put(Signal.MENTORING, List.of("Hire and Develop the Best"));
    m.put(Signal.CUSTOMER_FOCUS, List.of("Customer Obsession"));
    m.put(Signal.PRIORITIZATION, List.of("Bias for Action", "Frugality"));
    m.put(Signal.INCIDENT, List.of("Ownership", "Dive Deep", "Bias for Action"));
    return m;
}

static void leadershipPrinciples() {
    System.out.println();
    System.out.println("== 5. Amazon Leadership Principles covered by the bank ==");
    check(AMAZON_LPS.size() == 16, "Amazon lists 16 Leadership Principles");
    Map<Signal, List<String>> mapping = lpMapping();
    Map<String, Integer> storiesPerLp = new LinkedHashMap<>();
    for (String lp : AMAZON_LPS) {
        storiesPerLp.put(lp, 0);
    }
    for (Story story : sampleBank()) {

```

JAVA (continued 11/11)

```
Set<String> lps = new java.util.TreeSet<>();
for (Signal s : story.signals()) {
    lps.addAll(mapping.get(s));
}
for (String lp : lps) {
    check(storiesPerLp.containsKey(lp), "mapping uses real LP names: " + lp);
    storiesPerLp.merge(lp, 1, Integer::sum);
}
}
List<String> uncovered = new ArrayList<>();
for (Map.Entry<String, Integer> e : storiesPerLp.entrySet()) {
    System.out.printf("  %-44s %d\n", e.getKey(), e.getValue());
    if (e.getValue() == 0) {
        uncovered.add(e.getKey());
    }
}
System.out.printf("  no story yet for: %s\n", uncovered);
check(
    uncovered.equals(
        List.of(
            "Strive to be Earth's Best Employer",
            "Success and Scale Bring Broad Responsibility")),
    "the two newest, people-and-impact LPs need their own stories");
}

public static void main(String[] args) {
    storyCoverage();
    loopPlanning();
    answerTiming();
    leadershipPrinciples();
    System.out.println();
    System.out.println("All behavioral-preparation checks passed.");
}
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: signals, levels, and mining your history for stories
- Interview Core: story cards, coverage, and STAR delivery
- SDE-2 Follow-up: conflict, failure, mentoring, and company-specific principles
- Challenge: a recorded mock round scored against the delivery targets

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

This is the first volume in the Interview Rounds segment.

[Series index](#)

[Return to the complete series index](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, System Design, or Interview Rounds first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design
System Design	SD 04	Low-Level Design and Machine Coding
System Design	SD 05	High-Level Design Case Book
Interview Rounds	IR 01	Behavioral and Leadership Interviews

All 44 publication editions are available now. Choose a segment first, then follow the books inside it in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Interview Rounds - Book 01 of 01 - Study Step IR-01. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.